Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Geçen dersin tekrarı: yazılımın tanımı, yazılım türleri, eski yazılım, yanılgılar, yazılım mühendisliği tanımı, CASE.
- Yazılım mühendisliğinin beslendiği bilim dalları ve yazılım sistemi kavramı.
- Dersin hedefleri: ders sonunda bilinmesi gerekenler.
- Klasik süreç modelleri: şelale (waterfall), prototip, artımlı (incremental), spiral, RAD, bileşen tabanlı (component-based), birleşik süreç (Unified Process).
- Çevik (agile) yaklaşım: ortaya çıkışı ve ilkeleri.
- Uç programlama (XP, Extreme Programming): planlama, tasarım, kodlama, test.
- Scrum: koşu (sprint), günlük toplantı, Scrum panosu ve diğer modellerle karşılaştırma.
- Model seçimi ve kuruma özgü uyarlamalar.

Hocanın Özellikle Vurguladığı Kısımlar

- **Süreç adımları her zaman sırayla gitmez.** Adımların nasıl dizildiği hangi modelin uygulandığını belirler.
- **Şelale modelinin en büyük dezavantajı:** Müşteri çalışan ürünü ancak en sonda görür. Ara aşamalarda yalnızca raporlar ve modeller gider.
- **Spiral model**, şelale, prototip ve artımlı modelin birleşimidir. Her çevrimde risk analizi vardır ve çok belge üretir. Bu yüzden özellikle büyük ve hata kritik (safety-critical) projelerde kullanılır.
- **Çevik modellerin temeli:** müşterinin çalışan yazılımı erken görmesi, müşterinin ekip içinde olması, dinamik ekip ve basitlik.
- **"Çevik her projede başarılıdır" diye bir şey yok.** Model; projenin tipine, ekibin yapısına ve yazılımın konusuna göre seçilir. Slayttaki tablo Scrum tarafından bakılarak hazırlanmış.
- **XP'ye özgü uygulamalar:** kullanıcı hikâyeleri ve kabul testi kriterleri, basit tasarım ve CRC kartları, ikili programlama (pair programming), yeniden düzenleme (refactoring), sürekli bütünleştirme (continuous integration).
- **İkili programlama Scrum'da yok, XP'ye özgü.**
- Bugün Scrum en çok kullanılan çevik modeldir. Büyük firmalarda ve askeri yazılımlarda bile kullanılıyor.
- Kurumlar bir modeli olduğu gibi almak yerine kendilerine uygun bir model tanımlayıp ona uymalı.

Kısa Tekrar Notları

- **Yazılım mühendisliğinin beslendiği alanlar:** bilgisayar bilimi, yönetim bilimi (proje ve kalite yönetimi), ekonomi (maliyet, ekonomik fizibilite), bilişim.
- **Yazılım mühendisliği önce "yazılım ekonomisi" olarak başladı.**
- **Yazılım sistemi** tek başına düşünülmez. Kurumun diğer bilgi sistemleri, donanımı ve altyapısı da bilinmelidir.
- **Şelale modeli** (doğrusal model, ardışıl süreç, linear akış): gereksinim → tasarım → gerçekleme → bütünleştirme → teslim → işletim ve bakım. Her adım bitince diğerine geçilir. Gerektiğinde önceki adıma dönülebilir.

- **Prototip model:** Müşteriyle birlikte kolay araçlarla hızlı bir prototip hazırlanır. Prototip üzerinde anlaşınca asıl sistemin analizi ve tasarımı yapılır.
- **Artımlı model:** Önce çekirdek modül teslim edilir. Her iterasyon yeni bir artım (increment) ekler. Hangi modülün önce geleceği baştan planlanmalıdır.
- **Spiral model:** İçteki çevrim prototip, ikinci çevrim asıl ürün, sonraki çevrimler zenginleştirme ve bakım içindir.
- **RAD (Rapid Application Development):** Bağımsız modüllere ayrılabilen işi aynı anda farklı ekipler geliştirir, sonra modüller birleştirilir.
- **Bileşen tabanlı model:** Kurumun bileşen kütüphanesinden uygun bileşenler seçilip uyarlanır ve birleştirilir. Yeniden kullanımı (reusability) teşvik eder.
- **Unified Process fazları:** başlangıç (inception), detaylandırma (elaboration), inşa (construction), geçiş (transition).
- **Çevik ilkeler 12 maddedir.** Hoca bunları 4 temel fikirde topladı.
- **XP döngüsü:** planlama → tasarım → kodlama → test → artımsal sürüm (release).
- **CRC kartı:** Class (sınıf), Responsibility (sorumluluk), Collaborator (işbirliği yapılan sınıflar).
- **Scrum:** Görev listesi (backlog) → öncelikli maddeler için 1-4 haftalık koşu (sprint) → her gün 15 dakikalık toplantı.

Detaylı Açıklamalar

Geçen Dersin Tekrarı

- Yazılım yalnızca program değildir. Kullanıcı belgeleri, bakım için iç yapı ve veri yapısı belgeleri, yapım aşaması belgeleri ve test verileri de yazılımın elemanlarıdır.
- Yazılım türleri, ürün hattı, COTS, kurumsal yazılım ve eski yazılım (legacy software) tekrar edildi.
- Yanılgılar tekrar edildi: müşteri, yönetim ve geliştirici açısından.
- Yazılım projelerinin hâlâ yaklaşık yarısının başarısız olduğu söylendi.
- Mühendislikte ölçme ve tasarım, yazılım mühendisliği tanımı ve CASE araçları tekrar edildi.
- **Şemsiye aktiviteleri (umbrella activities):** Pressman'ın kitabındaki terim. Temel adımları destekleyen proje planlama, kalite yönetimi, yazılım ölçümü, risk analizi ve konfigürasyon yönetimi. Bunlar da süreç modellerinin içinde yer alır.
- **Bütünleşik CASE'in bileşenleri:**
  - Grafik arayüz.
  - Veri modellerini, veri sözlüğünü ve işlemleri tutan bir veri tabanı (repository).
  - Diyagram çizim araçları.
  - Analiz ve tasarımı ilişkilendirme yeteneği.
  - Kod üreticiler ve kod çeviriciler.
- **Analiz ve tasarımın nesneye yönelik akışı:**
  1. Kullanıcı hikâyeleri ve use case diyagramları.
  2. Sınıf diyagramları.
  3. Veri tabanına geçerken ER diyagramları.
  4. Tasarımda ayrıntılı sıralama (ardışıl) diyagramları ve kontrol akışı için durum diyagramları.
  5. Kodlama, veri tabanı ve tabloların oluşturulması.
  6. Test.
- **Yazılım konfigürasyon yönetimi:** Yazılımın elemanlarının ve aralarındaki ilişkilerin, kullanım ve geliştirme süreci boyunca nasıl değiştiğinin takibi. Sürüm kontrolü (version control) de bunun içindedir.

Dersin Hedefleri

Ders sonunda şunlar biliniyor olmalı:

- Yazılım mühendisliği ile programcılık arasındaki fark.
- Yazılım geliştirme süreci ve süreç modelleri.
- Yazılım yeterlilik olgunluk modeli ve sertifikasyon, yazılım standartları.
- CASE teknolojisi.

- Proje maliyeti ve maliyet tahmin yöntemleri, yazılım ölçüm kriterleri, proje planı.
- Kalite özellikleri ve kalite yönetim teknikleri.
- Bakım (software maintenance) ve konfigürasyon yönetimi (Software Configuration Management).
- Gereksinim analizi, gereksinim belirtimi (specification), tasarım süreci ve tasarım gösterimleri. Bunların nesneye yönelik olarak uygulanması.
- Tasarım kalıpları (design patterns).

Hoca, UML diyagramlarının nasıl çizildiğinin ve nesneye yönelik kavramların bilindiğinin varsayılacağını söyledi. Eksik olanlar Sistem Analizi ve Nesneye Yönelik Programlama derslerinin kaynaklarına bakmalı.

Klasik Süreç Modelleri

Şelale (Waterfall) modeli:

- Adımlar: gereksinim analizi → tasarım → gerçekleme → bütünleştirme (integration) → teslim (deployment) → işletim ve bakım.
- Gerçekleme içinde mühendislik testi (birim testi, bütünlük testi) yapılır. Sonra kullanıcı testi, kullanıma hazırlık (release) ve teslim gelir.
- Gereksinim analizi bitince bir gereksinim raporu, tasarım bitince bir tasarım raporu hazırlanır. Sonra bir sonraki adıma geçilir.
- Bir adımda sorun çıkarsa önceki adıma, hatta kullanıcıya kadar geri dönülebilir.
- **Dezavantaj:** Müşteri ürünü çalışır halde ancak en sonda görür. Kullanacağı ürünü zihninde canlandırması zaman alır. İstekleri ancak sonda gelir.

Prototip model:

- Şelalenin yavaşlığını gidermek için önerildi.
- Analiz, tasarım ve gerçekleme müşteriye sürekli danışılarak, daha kolay yazılım araçlarıyla bir prototip olarak yapılır.
- Prototipte anlaşınca asıl sistemin analizi, tasarımı ve bütünleştirmesi yapılır.

Artımlı (Incremental) model:

- Yazılım bir çekirdek ve etrafındaki modüller halinde planlanır.
- Her iterasyon sonunda yeni bir artım (increment) teslim edilir.
- **Örnek:** Öğrenci işleri yazılımı.
  1. Önce yeni kayıt olan öğrencilerin bilgilerinin alınıp veri tabanına aktarıldığı modül.
  2. Sonra transkript ve diğer belgeler.
  3. Sonra diğer çevre modüller.
- İlk önceliğin hangi modüle verileceği ve sonraki artımlar baştan planlanmalıdır. Müşteri yazılımı hızlı kullanmak istiyorsa uygundur, ama her projeye uymaz.

Spiral model:

- Hem iteratif hem evrimsel bir modeldir.
- **Çevrimler:**
  - En içteki çevrim: prototip.
  - İkinci çevrim: kullanılacak asıl ürün.
  - Sonraki çevrimler: ürünü zenginleştirme ve bakım.
- **Her çevrimde aynı dikey aşamalar tekrarlanır:**
  1. Müşteriyle iletişim: müşteri dinlenir, ne istediği anlaşılır.
  2. Planlama.
  3. Risk analizi.
  4. Mühendislik: analiz ve tasarım.
  5. Yapım (construction): kodlama, birim testi, kullanıcı testi.
  6. Teslim ve müşteri değerlendirmesi.
- Kullanıma hazırlıkta veri tabanlarına ilk bilgilerin yerleştirilmesi ve kullanıcı eğitimi de yapılır.
- Gerekli tüm belgelerin üretilmesi şart koşulur.
- Hızlı yazılım çıkarmak isteyen rekabetçi yazılım evlerine uygun değildir. Hata kritik yazılımlarda kullanılır.

RAD (Rapid Application Development):

- Hızlı yazılım ihtiyacı olan kurumlar için, kısa geliştirme çevrimli artımsal bir modeldir.
- **Ön koşullar:**
  - Uygulama, kısa sürede geliştirilebilecek, birbirinden görece bağımsız modüllere ayrılabilmeli.
  - Bileşen mimarisine uygun olmalı.
  - Her modül için eş zamanlı çalışacak ayrı ekipler bulunmalı.
- Çok yüksek teknik risk varsa uygun değildir.
- Genellikle dördüncü kuşak hızlı uygulama geliştirme araçları kullanılır.
- Artımlı modelden farkı: Modüller sırayla değil, aynı anda ayrı ekiplerce geliştirilir.

Bileşen tabanlı (Component-based) geliştirme:

1. Konu alanı (domain) anlaşılır.
2. Kurumun bileşen kütüphanesindeki uygun bileşenler seçilir.
3. Seçilen bileşenler yazılıma uyarlanır ve birleştirilir.

- Yazılım evinin zengin bir bileşen kütüphanesi olmalıdır.
- Yeniden kullanımı (reusability) teşvik eder. Reusability; yazılımın kendisinin, modüllerinin ya da çözüm yolunun tekrar kullanılabilmesidir. Son 15 yılda çok önem kazandı.
- RAD ekipleri de bileşen mimarisiyle daha hızlı üretim yapar.

Unified Process (Rational Unified Process, RUP):

- Rational firmasının bütünleşik CASE aracının altındaki süreç modeli. Rational'ı sonradan IBM satın aldı. Bugün IBM'de bir platform olarak, yazılım geliştirmenin her aşamasını destekliyor.
- Nesneye yönelik yazılım geliştirmeye uygun olduğu bilinir.
- Spiral modeldeki iletişim, planlama ve risk analizi adımlarını farklı fazlarda birleştirir:

Faz	Yapılanlar
Başlangıç (Inception)	Vizyon, başlangıç use case modeli, başlangıç proje sözlüğü, iş süreci, risk durumu
Detaylandırma (Elaboration)	Use case modelinin tanımlanması, analiz, yazılım mimarisi, ön tasarım, risk listesi, proje planının son hali
İnşa (Construction)	Tasarım modeli, artımlarla bütünleşik yazılım, test planı ve test case'ler, destek belgeleri, kullanıcı el kitapları
Geçiş (Transition)	Teslim (deployment) ve sürüm (release): kullanıma hazırlık ve kullanıma verme

- Sürüm yönetimi (release management) bu modelle önem kazandı.
- Hoca bu modeli klasik modellerden çevik modellere geçiş modeli olarak niteledi. Belgeleme vardır, ama spiraldeki gibi tekrar tekrar döngüler yoktur.

Çevik (Agile) Yaklaşım

- 2000'lerin başında belgeler ve standartlar yazılım geliştiricileri boğmaya başladı. Bunun üzerine bir manifesto yayımlandı.
- "Bizim işimiz kısa sürede yazılım elde etmek, yavaşlatan etkenleri gözden geçirmeliyiz" fikriyle çevik süreçler doğdu.
- Türkiye'de kavramlar 2003 civarında geldi. Önce yalnızca konferanslarda tanıtıldı, şirketlerde uygulama sonra başladı.
- "Çevik model" tek bir modelin adı değildir. Çevik yaklaşıma uygun süreç modellerinin ortak adıdır.
- **Çevik ilkelerde öne çıkanlar:**
  - Müşteri memnuniyetini erken sağlamak. Kullanılabilir yazılımı erken ve sık teslim etmek.
  - Müşteri temsilcisini ekibe dahil etmek. Müşteri ve geliştiriciler projede her gün birlikte çalışır.

  - Motive edici bir çalışma ortamı.
  - Raporlar yerine yüz yüze görüşmeler.
  - İlerlemenin ölçüsü çalışan yazılımdır.
  - Sponsor (parayı ödeyen), geliştiriciler ve kullanıcıların tanımlı olması ve birbirini kontrol edebilmesi.
  - Teknik açıdan iyi ve çevik bir tasarım.
  - Basitlik: Karmaşık çözümler yerine basit çözümle başlayıp geliştirmek.
  - Ekibin düzenli aralıklarla kendini değerlendirmesi.
- **Hocanın özetlediği 4 temel fikir:**
  1. Müşteri üretilen kodu daha hızlı görür ve onun üzerinden değerlendirme yapar.
  2. Müşteri ekibin içindedir, sonuçları anında değerlendirir.
  3. Ekip içinde dinamik bir yapı vardır.
  4. Basitlik: basitten karmaşığa doğru ilerlenir.

Uç Programlama (XP, Extreme Programming)

- Adı "extreme" ve "programming" kelimelerinden gelir. Türkçede "aşırı programlama", "uç programlama" gibi adlar denendi, ama yaygın olarak XP denir.
- Artımlı bir modeldir. Döngü: planlama → tasarım → kodlama → test → artımsal sürüm.
- **Planlama:**
  - Kullanıcı hikâyeleri (user stories) alınır. Her kullanıcı işi kendi bakış açısından anlatır. Örnek: satış-fatura sürecinde kasadaki kişi ve stoktaki kişi.
  - Kabul testi kriterleri (acceptance test criteria) alınır: Yazılım bittiğinde özellikle neyin test edileceği.
  - İterasyon planı yapılır: Hangi modül önce geliştirilecek?
- **Tasarım:**
  - Basit tasarım yapılır.
  - CRC kartları hazırlanır. Her kart bir sınıfın adını, sorumluluklarını (hangi işleri yapacağı) ve işbirliği yapacağı sınıfları gösterir.
  - Zor kısımlar için prototip çözümler (spike solution) hazırlanır.
- **Kodlama:**
  - İkili programlama (pair programming): Bir kod parçasını iki geliştirici birlikte yazar.
  - Yeniden düzenleme (refactoring): Mevcut kodun davranışı değiştirilmeden iyileştirilmesi.
- **Test:**
  - Kodu yazılan her birim için birim testi yapılır.
  - Sürekli bütünleştirme (continuous integration): Her birim bitince bütüne eklenip bütünlük testi yapılır.
  - Kabul testi, planlamadaki kriterlere göre yapılır.
  - Müşteri artımsal ürünü dener. Sonra sürüm (release) çıkarılır ve proje hızı (velocity) hesaplanır.
- **Test güdümlü geliştirme (TDD, Test-Driven Development):** Hoca bunun ayrı bir süreç modeli değil, çoğu çevik modelde bulunan bir yaklaşım olduğunu söyledi.

Test sırası (ileride ayrıntılı görülecek): birim testi → bütünlük testi → sistem testi → kullanıcı (kabul) testi.

Scrum

- Hocaya göre adı, (Amerikan futbolunda) oyuncuların kısa bir taktik toplantısı için bir araya gelmesinden geliyor. Terim büyük olasılıkla ragbiden gelir (bkz. ders sonrası notlar). Yazılım geliştirme de bir kazanma koşusu gibi düşünülmüş.
- **İşleyiş:**
  1. Yapılacak iş alt görevlere bölünür. Görev listesi kullanıcı hikâyelerinden çıkar ve önceliklendirilir.
  2. Listeden bir madde seçilir ve 1-4 haftalık bir koşuda (sprint) gerçekleştirilir.
  3. Koşu boyunca ekip her gün 15 dakikalık kısa bir toplantı yapar.
- **Günlük toplantının üç sorusu:**
  1. Son toplantıdan bu yana ne yaptınız?
  2. Hangi engellerle karşılaştınız?

3. Bir sonraki toplantıya kadar neyi başarmayı hedefliyorsunuz?

- **Scrum panosu (Scrum board):** Hangi işlerin yapılacağı, hangilerinin yapıldığı işaretlenir. Proje yöneticisinin tek tek sormasına gerek kalmaz, herkes birbirini denetler.
- Müşteri de ekiple birlikte çalışır, yapılanları değerlendirir ve kontrol eder.
- Son ürün proje sırasında şekillenir. Maliyet başta kabaca belirlenir, plan sürekli güncellenir.
- Danışmanlar yalnızca eğitimde değil, gerekirse ekibin üyesi olarak çalışır.

Scrum ve diğer modeller (tablo büyük olasılıkla slayttan; altyazıda hücreler okunmuyor):

Kriter	Waterfall	Spiral	Iterative	Scrum
Tanımlı süreçler	Gerekli	Gerekli	Gerekli	Planlama ve kapatma aşamasında
Final ürün	Planlamada tanımlanmış	Planlamada tanımlanmış	Proje esnasında şekilleniyor	Proje esnasında şekilleniyor
Proje maliyeti	Planlama esnasında hesaplanıyor	Kısmen değişken	Proje esnasında şekilleniyor	Proje esnasında şekilleniyor
Tamamlanma tarihi	Planlama esnasında belirleniyor	Kısmen değişken	Proje esnasında şekilleniyor	Proje esnasında şekilleniyor
Dış çevreye duyarlılık	Sadece planlamada	Planlama öncelikli	İterasyon sonlarında	Her zaman
Takım esnekliği, yaratıcılığı	Limitli, fazla yaratıcılık ve esneklik desteklenmiyor	Limitli	Limitli	İterasyonlar boyunca yaratıcılıkta ve esneklikte limit yok
Bilirkişi desteği (harici)	Proje eğitimi esnasında	Proje eğitimi esnasında	Proje eğitimi esnasında	Gerekirse proje içinde takıma bile dahil edilebilir
Başarı olasılığı	Düşük	Orta düşüklükte	Orta seviye	Yüksek

- Hoca bu tablonun Scrum tarafından bakılarak hazırlandığını vurguladı.

Model Seçimi ve Uyarlama

- Kanban da bir çevik yöntemdir. Bu derste üzerinde durulmadı.
- Scrum'ı önce küçük yazılım evleri kullanmaya başladı. Bugün büyük firmalarda, hatta askeri ve hata kritik yazılımlarda bile kullanılıyor.
- Yazılım mühendisliği sempozyumlarında Scrum'ın gerçekten ne kadar fayda sağladığı hâlâ tartışılıyor.
- Bazı kurumlar çeviği kendilerine uyarlıyor. Hoca, büyük bir Türk şirketinin yaklaşık 5-6 yıl önce kendine uyarladığı bir çevik modeli örnek verdi.
- Önemli olan ekibin kendini disipline etmesi ve müşterinin isteklerini gözden kaçırmamasıdır. Kurum, analiz, tasarım, kodlama, test ve şemsiye aktivitelerini kapsayan, kendisine en uygun modeli tanımlamalıdır.

Sonraki Hafta

- Yazılım kurumlarında sertifikasyon: CMM ve CMMI (yazılım evinin yeterliliği).
- Süreç adımlarına ek açıklamalar.
- Gereksinim analizi.

Ders sonrası notlar

- **Scrum teriminin kökeni ragbidir (rugby), Amerikan futbolu değil.** Scrum, ragbide oyunu yeniden başlatmak için oyuncuların kenetlenerek oluşturduğu toplu düzendir. Terimi yazılım için büyük olasılıkla 1986'da Takeuchi ve Nonaka kullandı. Scrum çerçevesini 1995'te Schwaber ve Sutherland tanıttı.
- **Scrum'da roller:** Hocanın "proje lideri / proje koçu" dediği kişinin resmi adı Scrum Master'dır. Müşterinin ekipteki temsilcisi Product Owner'dır. Görev listesine Product Backlog denir.
- **Agile Manifesto** 2001'de yayımlandı. 12 ilkenin üstünde 4 temel değer vardır:
  1. Süreçler ve araçlar yerine bireyler ve etkileşimler.
  2. Kapsamlı belgeler yerine çalışan yazılım.
  3. Sözleşme pazarlığı yerine müşteriyle işbirliği.
  4. Plana bağlı kalmak yerine değişime yanıt vermek.

  Hocanın 4 maddesi bunların bir yorumudur.
- **XP "ilk çevik model" sayılmaz.** XP'yi büyük olasılıkla Kent Beck 1996-1999 arasında geliştirdi. Scrum büyük olasılıkla 1995'te tanıtılmıştı. İkisi de manifestodan öncedir.
- **İkili programlamada** iki kişinin rolü: Biri kodu yazar (driver), diğeri anlık olarak gözden geçirir ve yön verir (navigator). Roller sık sık değişir. Hocanın "biri yazarken diğeri test eder" demesi, bu gözden geçirme rolünü anlatıyor.
- **TDD'de testler koddan önce yazılır.** Önce başarısız olan bir test yazılır, sonra testi geçecek kadar kod yazılır, sonra yeniden düzenleme yapılır (red-green-refactor). Hoca TDD'yi daha çok kabul testi kriterlerinin baştan alınması yönüyle anlattı.
- **CRC'nin C'si "Collaborator"dır.** Kartta sınıfın işbirliği yaptığı diğer sınıflar yazılır. Ayrı bir "collaboration kartı" yoktur.
- **Unified Process fazlarının Türkçesi:** Inception = başlangıç, elaboration = detaylandırma/ayrıntılandırma. Hoca elaboration için "başlangıç ve hazırlık aşaması" dedi, bu karışıklık yaratabilir. RUP'u geliştiren Rational Software'i IBM büyük olasılıkla 2003'te satın aldı.
- **Model kökenleri (büyük olasılıkla):**
  - Şelale modeli genellikle Winston Royce'un 1970 tarihli makalesine dayandırılır.
  - Spiral modeli Barry Boehm 1986-1988'de önerdi.
  - RAD'ı James Martin 1991'de kitaplaştırdı. Hoca RAD'ı "IBM'in önerdiği model" diye tanıttı; Martin'in çalışmaları büyük olasılıkla IBM'deki işlerine dayanır.
- **Altyazıdaki bozuk yerler:**
  - "RAT model" = RAD model.
  - "ekremental" = incremental.
  - "ecel / ECR" = agile.
  - "bilim testi" = birim testi.
  - "Türksel'de T-Ecel" kurumun kendine uyarladığı bir çevik modeli anlatıyor; adı altyazıdan kesin okunamıyor.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.